

Production Deployment Setup

Bihar Textbook Backend · GitHub Actions self-hosted runner on Windows Server · September 2026

How deployment works

Developers push their work to the **main** branch. Pushing to main never touches the server. When a release is ready, main is promoted to the **master** branch. Every push to master starts the **Deploy Backend to Production** workflow, which runs on a GitHub Actions runner installed on the production Windows Server.

```
feature work --push--> main          (nothing is deployed)
                        |
                        | promote when ready
                        v
                    master --push--> GitHub Actions --> runner on Windows Server
                                                checkout -> npm ci ->
                                                prisma generate -> pm2 restart
```

Already done

- Workflow file `.github/workflows/deploy.yml` is committed. It runs only on pushes to master.
- The master branch exists, matches main, and is the repository's default branch on GitHub.
- `.env`, `node_modules`, `/src/generated/prisma` and `/uploads` are excluded from Git. No secrets are in the repository.

Still to do

Steps 0 to 6 below. Step 0 is a code change made by a developer. Steps 1 to 6 happen on GitHub and on the server. Do them in order.

Step 0 — Update the workflow (developer, before Step 4)

Status: not yet applied. The workflow as committed would cause two problems on Windows. Apply this change before the first real deploy.

The app reads `.env` from its working folder and saves uploaded files to `<working folder>\uploads`. This causes two problems:

- **Uploads and `.env` would be deleted on every deploy.** By default, `actions/checkout` deletes all files Git ignores in the runner folder. Setting `clean: false` stops this.
- **Deploys would fail with `EPERM` errors.** Windows locks files that a running program has loaded. While the app is running, `npm ci` cannot replace `node_modules` and `prisma generate` cannot replace the Prisma engine DLL. The app must be stopped first.

Corrected steps section of `.github/workflows/deploy.yml`:

```
steps:
  - name: Checkout production code
    uses: actions/checkout@v4
    with:
      clean: false

  - name: Stop PM2 application
    run: pm2 stop my-backend
    continue-on-error: true

  - name: Install dependencies
    run: npm ci

  - name: Generate Prisma Client
    run: npx prisma generate

  - name: Restart PM2 application
    run: pm2 restart my-backend
```

Commit to main, then promote to master. The site is down for a few seconds during each deploy.

Step 1 — Cancel the queued run

Pushing master for the first time started a deploy that is waiting for a runner. Cancel it so it does not start the moment the runner is registered.

- GitHub → repository → **Actions** → **Deploy Backend to Production** → open the queued run → **Cancel workflow**.
- Or from a developer machine: `gh run cancel 35733323315`

Step 2 — Check the server

Connect to the server over Remote Desktop and open **PowerShell as Administrator**.

```
node -v; npm -v; git --version; pm2 -v
pm2 list
pm2 describe my-backend      # note "exec cwd" (current app folder) and the process name
whoami                       # note this user - the runner must run as this same user
```

- If git is missing, install Git for Windows. The checkout step needs it.
- If the PM2 process is not named my-backend, either rename it in Step 5 (recommended) or ask the developer to update the name in the workflow.
- Write down the **exec cwd** path. Step 5 calls it \$OLD.

Step 3 — Install the GitHub runner

1. On GitHub: repository → **Settings** → **Actions** → **Runners** → **New self-hosted runner**. Choose **Windows** and **x64**.
2. The page shows the exact commands, including the current version and a one-time token. Run them in PowerShell. They look like this:

```
mkdir C:\actions-runner; cd C:\actions-runner
Invoke-WebRequest -Uri <url from page> -OutFile actions-runner-win-x64-<ver>.zip
Add-Type -AssemblyName System.IO.Compression.FileSystem
[System.IO.Compression.ZipFile]::ExtractToDirectory("$PWD\actions-runner-win-x64-<ver>.zip", "$PWD")
.\config.cmd --url https://github.com/nsappsinnovation/Bihar-Textbook-Backend --token <TOKEN>
```

3. Answer the `config.cmd` prompts:

Prompt	Answer
Runner group	Press Enter (default)
Runner name	Press Enter, or e.g. prod-win
Labels	Press Enter (default)
Work folder	Press Enter (_work)
Run as service?	Y
User account for the service	The same user shown by whoami in Step 2 (e.g. .Administrator), then its password. Do not accept the default NETWORK SERVICE.

Why the user account matters: PM2 keeps its process list per Windows user (in `C:\Users\<user>\.pm2`). If the runner runs as a different user, pm2 restart will not find the app, and node or pm2 may not be found at all.

4. Confirm the runner shows as **Idle** (green) on the Runners page.

Step 4 — First deploy (puts the code on the server)

Make sure Step 0 is done. In the **Actions** tab, open the latest run and choose **Re-run all jobs**. The code is checked out to:

```
C:\actions-runner\_work\Bihar-Textbook-Backend\Bihar-Textbook-Backend
```

The PM2 step may still restart the old app from its old folder. That is expected at this stage.

Step 5 — Run the app from the runner folder (one-time move)

From now on, deploys update the runner folder, so PM2 must run the app from there. Replace the `$OLD` path with the `exec cwd` from Step 2.

```
$W = "C:\actions-runner\_work\Bihar-Textbook-Backend\Bihar-Textbook-Backend"
$OLD = "C:\path\from\step2"

Copy-Item "$OLD\.env" "$W\.env"
robocopy "$OLD\uploads" "$W\uploads" /E

cd $W
pm2 delete my-backend
pm2 start server.js --name my-backend # not "npm start" - that runs nodemon
pm2 save
```

- Check that the API responds and that existing uploaded images load (`/uploads/images/...`).
- Keep the old folder for a few days as a backup, then remove it.

Step 6 — Test a full release

On a developer machine, make a small change on main, then promote it:

```
git checkout main # commit and push a small change here first
git checkout master && git pull
git merge --ff-only main
git push origin master # this push deploys
git checkout main
```

- In the Actions tab, every step should turn green.
- On the server, `pm2 list` should show `my-backend` online with a higher restart count.
- `Test-Path "$W\.env"` should still return **True**.

Ongoing operations

Topic	What to do
Releasing	Always work on main, then fast-forward master to main and push master (Step 6). Never commit directly to master. Optionally, protect master in GitHub so it only accepts pull requests.
Database schema changes	Migrations are not run by the workflow. After changing prisma/schema.prisma, run <code>npx prisma migrate deploy</code> in \$W on the server, before or right after promoting.
Changing .env	Edit \$W/.env on the server, then run <code>pm2 restart my-backend</code> . Never commit .env to Git.
Server reboot	<code>pm2 save</code> alone does not survive a reboot on Windows. Install <code>pm2-installer</code> (or <code>pm2-windows-startup</code>) under the same user.
Rolling back	Reset master to the previous good commit and push with <code>--force-with-lease</code> , or revert on main and promote again.

Troubleshooting

Symptom in the Actions log	Fix
pm2 : The term 'pm2' is not recognized	The runner service user does not have npm's global folder on its PATH. Fix PATH, then run <code>Restart-Service "actions.runner.*"</code> .
Process or Namespace my-backend not found	The runner runs as a different Windows user than PM2, or the process name differs. Recheck Steps 3 and 5.
EPERM ... query_engine-windows.dll.node, or EPERM inside node_modules	The running app is locking the files. Apply the Step 0 change (stop the app before install).
Job stays Queued	The runner is offline. Check with <code>Get-Service "actions.runner.*"</code> .
Uploads or .env missing after a deploy	<code>clean: false</code> is missing from the checkout step (Step 0). Restore the files from the old folder or a backup.